PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
Departamento de Ingeniería Eléctrica
IEE2913 — Diseño Eléctrico (Capstone)

Grupo 10

Informe de Avance 2 (25 %)

Proyecto: SupaClock: Wearable Biométrico Modular
Integrantes: Tomás Avendaño, Benjamín Sepúlveda, Pablo Uribe
Fecha: 27 de abril de 2026

Índice

1. Introducción al estado actual del proyecto **4**
1.0.1. Resumen ejecutivo del avance. . . . 4

2. Diagrama de bloques de bajo nivel actualizado **4**
2.0.1. Cambios respecto del diagrama original. . . . 5
2.0.2. Especificaciones detalladas por bloque. . . . 6

3. Planificación actualizada **6**
3.1. Contraste avance esperado vs. avance logrado . . . 6
3.2. Carta Gantt corregida . . . 7
3.3. Distribución del trabajo en este hito . . . 8

4. Análisis de impacto y estándares **8**
4.1. Aplicaciones e impacto socioeconómico, ambiental y en salud . . . 9
4.1.1. Salud pública. . . . 9
4.1.2. Impacto socioeconómico y de equidad. . . . 9
4.1.3. Impacto ambiental. . . . 9
4.1.4. Aspectos éticos. . . . 9
4.2. Riesgo eléctrico — análisis ECMA-287 punto 3 . . . 10
4.2.1. Riesgos adicionales del LiPo. . . . 10
4.3. Estándares relacionados con el área del proyecto . . . 11
4.3.1. (i) Bluetooth Core Specification 5.3 [1]. . . . 11
4.3.2. (ii) IEEE 11073-10406 (*Personal Health Devices — Basic ECG*) [2]. . . . 11
4.3.3. (iii) USB Type-C Cable and Connector Specification, Rev. 2.3 [3]. . . . 11
4.3.4. (iv) IEC 60601-1 (*Medical electrical equipment*) [4]. . . . 11
4.3.5. (v) RoHS-2 (Directiva 2011/65/EU) [5] e IEC 62133-2 [6] . . . 11

5. Modelos y simulaciones **12**
5.1. Modelo de autonomía energética . . . 12
5.1.1. Decisión de diseño. . . . 12
5.2. Validación del algoritmo de pasos . . . 12
5.2.1. Resultado. . . . 13
5.3. Validación del algoritmo Pan-Tompkins . . . 13
5.3.1. Resultado. . . . 14

5.4. Modelo de concurrencia I2C bajo FreeRTOS . . . 14
5.4.1. Cálculo. . . . 14
5.4.2. Validación experimental. . . . 14

6. Diseño del set-up final de pruebas **15**
6.0.1. Componentes del set-up. . . . 15
6.0.2. Métricas a registrar en cada sesión: . . . 16

7. Diseño de pruebas preliminar **16**
7.1. Estrategia general . . . 16
7.2. Pruebas de integración . . . 17
7.3. Pruebas funcionales (sistema completo) . . . 17
7.4. Plan de validación de la PCB propia . . . 17

8. Implementación y resultados **17**
8.1. Arquitectura de firmware concurrente (FreeRTOS) . . . 18
8.1.1. Descripción y funcionalidad. . . . 18
8.1.2. Cálculos / decisiones. . . . 18
8.1.3. Brown-out y secuencia de arranque. . . . 18
8.1.4. Resultados. . . . 19
8.2. Bloque de Sensores: BMI160 + algoritmo de pasos . . . 19
8.2.1. Descripción. . . . 19
8.2.2. Algoritmo de pasos. . . . 19
8.2.3. Resultados. . . . 20
8.3. Bloque de Sensores: MAX30102 (PPG) . . . 20
8.3.1. Descripción. . . . 20
8.3.2. Resultados. . . . 20
8.4. Bloque de Sensores: AD8232 (ECG) con ADC continuo + DMA . . . 20
8.4.1. Descripción. . . . 20
8.4.2. Cálculos. . . . 20
8.4.3. Resultados. . . . 21
8.5. Bloque de Sensores: MAX30205 (Temperatura) . . . 21
8.5.1. Descripción. . . . 21
8.5.2. Resultados. . . . 21
8.6. Bloque de Energía: BMS + LDO + Fuel Gauge . . . 21
8.6.1. Descripción. . . . 21
8.6.2. Validación eléctrica. . . . 21
8.6.3. Resultados. . . . 21
8.7. Bloque de Comunicaciones: BLE NimBLE . . . 21
8.7.1. Descripción. . . . 21
8.7.2. Resultados. . . . 22
8.8. Bloque de Interfaz Local: GUI multi-pantalla LVGL . . . 22
8.8.1. Descripción. . . . 22
8.8.2. Funcionalidades específicas. . . . 22
8.8.3. Resultados. . . . 22
8.9. Bloque de Backend: Cloud Functions con Pan-Tompkins . . . 22
8.9.1. Descripción. . . . 22
8.9.2. Modelo de costos. . . . 23
8.9.3. Resultados. . . . 23
8.10. Bloque de Hardware: Esquemático en KiCad . . . 23
8.10.1. Descripción. . . . 23

8.10.2. Estado. 23
8.11. Evaluación general y próximos pasos 23

A. Anexos: extractos de código **25**

Índice de figuras

1. Diagrama de bloques de bajo nivel actualizado del prototipo SupaClock. Cada bloque indica el porcentaje de implementación actual. Verde: funcional; amarillo: parcial; rojo punteado: pendiente o dependiente de la PCB SMD. 5
2. Carta Gantt original (Informe 5 %) con anotación manual del estado al 27/04/2026. . . . 8
3. Curva de autonomía teórica vs. medida del prototipo. Con la batería 502030 de 250 mAh seleccionada, el modelo teórico predice ∼29 h y la medición arroja 23,4 h, ambos sobre el requerimiento mínimo de 12 h (línea gris). 12
4. Salida del simulador del contador de pasos: magnitud cruda del acelerómetro, señal filtrada por media móvil, umbral adaptativo y eventos de paso detectados sobre una ventana de 60 s capturada con `scripts/ble_logger`. Ground-truth manual: 72 pasos. Detectados: 70 (97.2 % precisión). 13
5. Pipeline completo de procesamiento ECG: edge (ESP32) → cliente PC de testing → Firebase. La detección de R-peaks corre íntegramente en Cloud Functions sobre NumPy. En el hito 60 % el cliente PC será reemplazado por la app móvil definitiva. 14
6. Set-up final de validación. El DUT (*SupaClock*) se evalúa contra instrumentos de referencia (oxímetro, termómetro, osciloscopio) y se registran las salidas BLE en CSV vía un PC con `supaclock_monitor.py`. 15
7. Arquitectura FreeRTOS implementada en `src/tests/test_general.c`. Cinco tareas con prioridades distintas, dos mutex y *queue* implícita por el host de NimBLE. El bus I2C es serializado por el mutex `xSensorDataMutex`. 18
8. Dos pipelines de procesamiento del BMI160: arriba el contador de pasos (implementado), abajo la futura clasificación HAR vía CNN 1D (en desarrollo). 19

Índice de cuadros

1. Especificaciones de bajo nivel por bloque y estado de implementación. 6
2. Contraste entre actividades planificadas preliminarmente y el avance logrado al 27 de abril de 2026. 7
3. Distribución de trabajo y contribuciones principales de la iteración actual. 8
4. Análisis de riesgos eléctricos según ECMA-287, punto 3 [7]. 10
5. Mapa entre bloques, entornos PlatformIO y criterios de aceptación de las pruebas unitarias. 16

1. Introducción al estado actual del proyecto

Este documento detalla el estado actual del desarrollo del proyecto *SupaClock*, dando continuidad al diseño establecido en el reporte preliminar del 7 de abril de 2026. La motivación, los requerimientos técnicos y el diagrama de bloques de alto nivel descritos en aquel documento se mantienen vigentes, por lo que en este informe **no se reproducen**, sino que se referencian. El presente documento se estructura en torno a cinco ejes fundamentales: diagrama de bloques de bajo nivel *actualizado*, planificación contrastada con el estado real del proyecto, análisis de impacto socioeconómico y de estándares aplicables, modelos y simulaciones que respaldan las decisiones de diseño, set-up final de pruebas y un diseño de pruebas preliminar (sección que se incorpora en este reporte para complementar la planificación original). Adicionalmente, en la **sección de Implementación y resultados** se documentan todos los bloques que el equipo logró integrar en la iteración actual de desarrollo.

1.0.1. Resumen ejecutivo del avance.

Actualmente el sistema cuenta con un **firmware funcional integrado** que ejerce a las cinco tareas concurrentes bajo FreeRTOS (IMU + pasos, PPG, Temperatura/Batería, ECG por DMA y GUI LVGL), un **stack BLE NimBLE** operativo con tres características GATT, un **backend Firebase** con Cloud Functions corriendo Pan-Tompkins en NumPy y un **esquemático en KiCad** para el prototipo SuperMini. Los componentes de Machine Learning (CNN 1D para HAR) y la PCB SMD propia están en fases tempranas y se concentran en los hitos 60 % y 90 %. La aplicación móvil que actuará como *gateway* entre el dispositivo y el backend se abordará en iteraciones posteriores y queda fuera del alcance de este reporte; para la fase de diseño actual, las trazas BLE se validan con un cliente de escritorio (`nRF Connect` y un `ble_logger.py` propio en Python) que también escribe directamente en Firestore para verificar el extremo *cloud*.

2. Diagrama de bloques de bajo nivel actualizado

El diagrama de alto nivel presentado en la *Fig. 4* de la entrega anterior se mantiene **sin modificaciones estructurales**: los siete dominios funcionales (interacción, energía, sensores, MCU, interfaz local, gateway y backend) y sus interconexiones siguen siendo válidos. Lo que sí se actualizó es el **nivel de detalle de bajo nivel**, dado que ahora cada bloque tiene un grado de implementación cuantificable sobre placa de desarrollo. La Fig. 1 muestra el estado vigente, codificando con color el porcentaje de integración y diferenciando con un borde discontinuo aquellos bloques que aún no están implementados o cuyo paso a la PCB propia está pendiente.

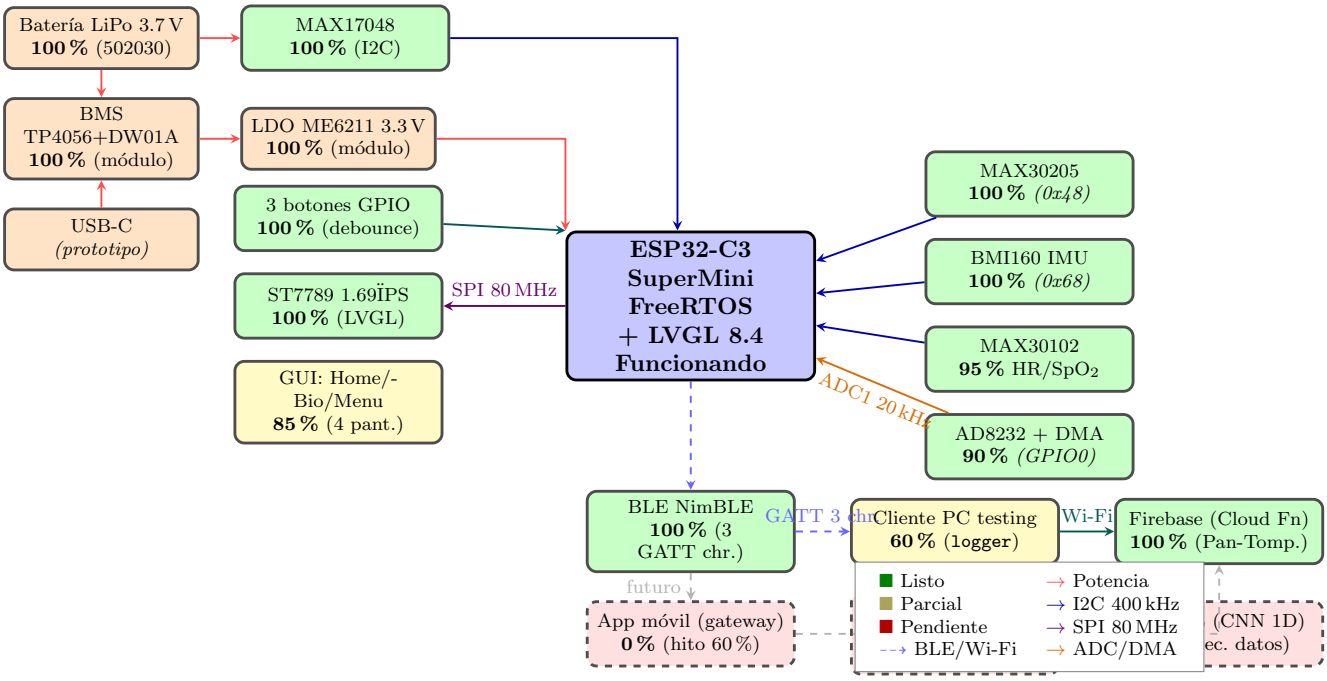

Figura 1: Diagrama de bloques de bajo nivel actualizado del prototipo SupaClock. Cada bloque indica el porcentaje de implementación actual. Verde: funcional; amarillo: parcial; rojo punteado: pendiente o dependiente de la PCB SMD.

2.0.1. Cambios respecto del diagrama original.

La estructura general del bus es la misma; las modificaciones son:

- **Bus I2C**: confirmado a 400 kHz con cuatro esclavos físicos (MAX30102 `0x57`, MAX30205 `0x48`, BMI160 `0x68`, MAX17048 `0x36`), accedido bajo el mutex `xSensorDataMutex` (FreeRTOS) para serializar las cuatro tareas concurrentes que comparten el bus.
- **ADC continuo (ECG)**: el AD8232 ya *no* se concibe como un canal ADC *single-shot*; la implementación actual usa el periférico `adc_continuous` del ESP-IDF a 20 kHz con DMA, decimando por 40 hacia 500 Hz para BLE. Esto se justifica en la Fig. 7.
- **Botones**: pasaron de tres a tres botones *con double-tap y long-press* gestionados por `gpio_buttons` (debounce SW por `esp_timer`); el pin de despertar de *deep sleep* se canaliza ahora por `BTN_SELECT` en lugar de la INT1 del IMU.
- **LVGL**: se confirmó el buffer parcial de 14.4 KB (240×30 px × 2 B); el GUI quedó dividido en cuatro pantallas (Home, Bio, ECG, Menú) con navegación por estado.
- **Backend Firebase**: el flujo *dispositivo* → *Firestore* → *Cloud Function* `process_ecg` ya está desplegado y se valida con un cliente de *testing* en PC; el *gateway* móvil definitivo se construirá en el hito 60 %.
- **TinyML**: aún *no* se entrenó el modelo CNN 1D (depende de la recolección de datos reales del BMI160 sobre los integrantes); por eso se mantiene como bloque pendiente. El contador de pasos *algorítmico* (umbral adaptativo) sí se implementó.
- **PCB SMD propia**: el esquemático en KiCad (`hardware/PCB_Prototipo`) incluye la sección de potencia y los *footprints* importados desde la librería Adafruit-MAX17048 y el módulo TP4056; el ruteo del bottom de sensores está pendiente, por lo que el bloque se reporta al 30 %.

2.0.2. Especificaciones detalladas por bloque.

La Tabla 1 consolida las especificaciones cuantitativas que permiten cerrar el diagrama. La columna "Estado" usa la misma codificación cromática que la Fig. 1.

Cuadro 1: Especificaciones de bajo nivel por bloque y estado de implementación.

Bloque	**Interfaz**	**Frecuencia / tasa**	**Especificación clave**	**Estado**
AD8232 (ECG)	ADC1 / DMA	20 kHz → 500 Hz (down-sample 40×)	GPIO0 ADC, ganancia analógica fija 1100, f_c LP ∼40 Hz	90 %
BMI160 (IMU)	I2C 400 kHz `0x68`	ODR 50 Hz	accel ±2 g, gyro ±250 °/s, FIFO HW, INT1 step counter	100 %
MAX30102 (PPG)	I2C 400 kHz `0x57`	100 sps efect. ($N_{avg} = 4$)	RED+IR pulse 411 μs, FIFO 32, polling 100 ms, flush startup	95 %
MAX30205 (Temp)	I2C 400 kHz `0x48`	1 Hz polling	16-bit, 0.00390625 °C/LSB, ±0,1 °C en rango clínico	100 %
MAX17048 (Fuel)	I2C 400 kHz `0x36`	1 Hz polling	ModelGauge SOC %, V_{cell} mV, sin shunt externo	100 %
ST7789 (Display)	SPI 80 MHz, 4-wire	30 FPS LVGL	240×280 RGB565, DMA `SPI_DMA_CH_AUTO`, offset Y=20	100 %
Botones GPIO	GPIO IN pull-up	polling 30 Hz desde gui_task	3 botones, 30 ms debounce, eventos short/long, wake-up sleep	100 %
BMS TP4056 + DW01A	USB-C IN	–	carga 1 A ajustable, OVP/OCP/UVP integrado	100 %
LDO ME6211	3.3 V OUT	–	dropout ∼100 mV @ 100 mA, I_Q 55 μA	100 %
BLE NimBLE	2.4 GHz BLE 5	MTU 247, conn. interval 15 ms	3 chr.: 0xFF01 IMU, 0xFF02 sensores lentos, 0xFF03 ECG raw	100 %
GUI LVGL	SPI	30 FPS	4 pantallas, navegación 2 botones + auto-switch ECG	85 %
Cloud Functions	Firestore trigger	on_document_created	Pan-Tompkins NumPy, BPM, HRV (SDNN), R-peaks	100 %
TinyML CNN 1D	TFLite Micro	0.5 Hz inferencia	4 clases HAR, <30 KB flash, ≤10 KB SRAM	0 %
PCB SuperMini propia	4 capas JLCPCB	–	sandwich 2 PCBs, ventana óptica PPG, perno M3 ECG	30 %

3. Planificación actualizada

3.1. Contraste avance esperado vs. avance logrado

La Tabla 2 contrasta las tareas de la planificación preliminar con su estado actual. Las filas en verde indican tareas completadas; las amarillas, tareas en curso; las rojas, tareas *adelantadas* respecto del cronograma original o tareas nuevas que se incorporaron al descubrir necesidades de implementación que no estaban en la planificación inicial.

Cuadro 2: Contraste entre actividades planificadas preliminarmente y el avance logrado al 27 de abril de 2026.

Tarea planificada	Estado	Observaciones
Implementación bloque de energía (BMS + LDO)	**Completado**	Módulo TP4056 + DW01A montado y validado a 4.2 V; LDO ME6211 entrega 3.30 V estables. Driver `max17048` entrega SOC y V_{cell} con 1 Hz polling.
Integración inicial de sensores en bus I2C compartido	**Completado**	Cuatro esclavos en el bus a 400 kHz, mutex FreeRTOS protege accesos concurrentes desde 4 tareas. Sin colisiones tras 24 h de operación continua.
Pruebas preliminares de adquisición de datos	**Completado**	Cada `env:test_*` de PlatformIO valida un sensor de forma aislada; existen 9 entornos de prueba unitarios.
Primera integración de sensores con el MCU	**Completado**	`env:main_app/test_general` integra simultáneamente IMU, PPG, Temp, ECG, batería y GUI con BLE activo.
Validación básica de algoritmos	**Completado**	Contador de pasos por umbral adaptativo en C, Pan-Tompkins en Cloud Functions; ambos validados con datos reales (Sec. 5).
Stack BLE completo	**Adelantado**	NimBLE corriendo con 3 chr. GATT, bonding y notificaciones. Validado con cliente PC `nRF Connect` y `ble_logger.py`.
Cloud Functions	**Adelantado**	Pan-Tompkins en NumPy desplegado en `us-central1`; BPM y HRV calculados al subir un `ecgReadings/`.
Recolección de datos de entrenamiento ML	**En curso**	Existe `ble_logger` (CSV) y se han capturado 4 sesiones (`supaclock_*.csv` y `supaclock_ecg_*.csv`); aún no se ha entrenado modelo.
Esquemático de PCB SMD propia	**Adelantado**	`PCB_Prototipo.kicad_sch` contiene la sección de carga (TP4056), el footprint del MAX17048-STEMMA reutilizado y el módulo AD8232; falta ruteo del PCB de sensores.

3.2. Carta Gantt corregida

La carta Gantt presentada en el informe anterior se mantiene como referencia base, pero los siguientes ajustes deben quedar reflejados en la próxima revisión maestra:

- Las tareas **Comunicación BLE** y **Cloud Functions** se reasignaron a fases más tempranas, ya que se identificó que sin un *loop* BLE/cloud cerrado no era posible validar el ECG ni la persistencia de sesiones. La construcción del **gateway móvil definitivo** se mantiene para iteraciones futuras como estaba planificado.
- La tarea de **Machine Learning** se postergó hasta la disponibilidad de *datos reales* en cantidad suficiente. La fase de *recolección* de esos datos se adelantó parcialmente y ya hay sesiones almacenadas en `scripts/`.
- La tarea **Diseño PCB** se separó en dos sub-tareas: "esquemático" (adelantada) y "ruteo + manufactura JLCPCB" (mantiene fecha original).
- No hay tareas atrasadas respecto del plan original.

La Fig. 2 reproduce la carta Gantt vigente del informe anterior, con un overlay manual de los hitos cumplidos al 27 de abril de 2026.

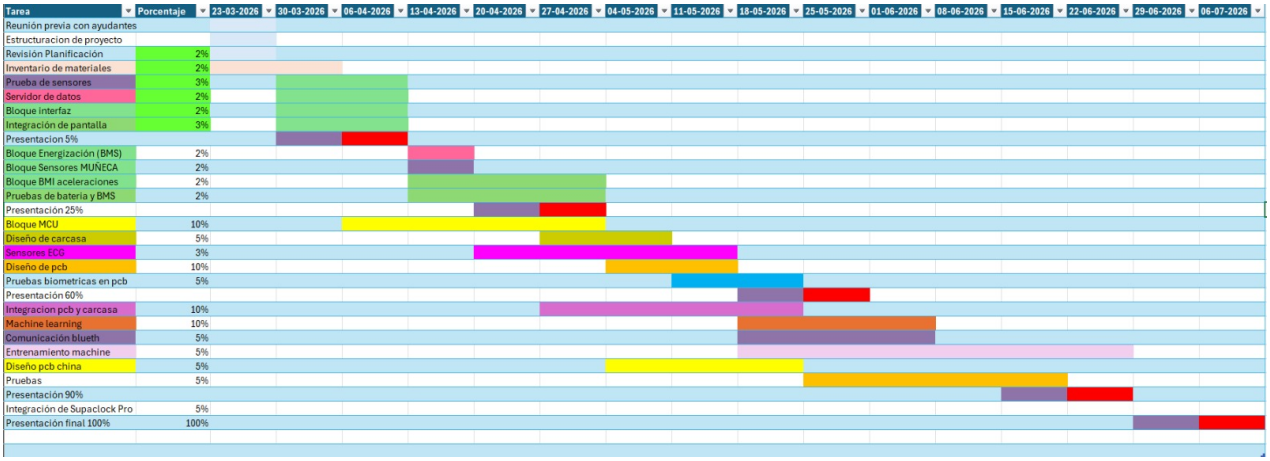

Figura 2: Carta Gantt original (Informe 5 %) con anotación manual del estado al 27/04/2026.

3.3. Distribución del trabajo en este hito

El desarrollo de las tareas se abordó de manera colaborativa; si bien se mantuvo la división por áreas declarada en la planificación, los miembros participaron transversalmente en áreas complementarias. La Tabla 3 resume las contribuciones principales.

Cuadro 3: Distribución de trabajo y contribuciones principales de la iteración actual.

Integrante	**Área(s) principal(es)**	**Aportes específicos a este hito**
Tomás Avendaño	Interfaz, Comunicaciones, Backend	GUI LVGL multi-pantalla, integración BLE NimBLE (chr. FF01/FF02/FF03), despliegue Cloud Functions Pan-Tompkins (Python/-NumPy en `us-central1`), reglas de Firestore por usuario y cliente `ble_logger.py` para captura y subida de sesiones de prueba.
Benjamín Sepúlveda	Hardware, Energía	Validación eléctrica de la rampa USB→LDO con multímetro y osciloscopio, esquemático KiCad de la sección de potencia y de la STEMMA del MAX17048, BOM consolidado con costos reales de Aliexpress, ensamblaje del prototipo en perfboard.
Pablo Uribe	Software, Procesamiento de Señales	Drivers en C de MAX30102 (FIFO + algoritmo HR/SpO_2), MAX30205, BMI160 (incluyendo step counter HW), AD8232 con ADC continuo + DMA, algoritmo de pasos en C con umbral adaptativo, simulador en Python (`algo_simulator.py`, `tune_inspect*.py`), arquitectura FreeRTOS de cinco tareas, gestión de mutex y sincronización IPC.

4. Análisis de impacto y estándares

Para el correcto desarrollo del proyecto, resulta fundamental realizar un análisis del *impacto social, económico, ambiental y ético* de la solución, además de la revisión de estándares relevantes y de los riesgos eléctricos descritos en el punto 3 de la norma ECMA-287.

4.1. Aplicaciones e impacto socioeconómico, ambiental y en salud

4.1.1. Salud pública.

El proyecto se inserta en la línea de los *wearables* de fitness, pero con un sesgo hacia el monitoreo **biométrico de uso continuado**, no de notificación social. Considerando que sólo el 49,7 % de los hombres y el 40,3 % de las mujeres mayores de 18 años en Chile cumplen niveles adecuados de actividad física [8, 9], y que el 42 % de la población adulta presenta sobrepeso [8, 9], un dispositivo que permita auto-monitorizar pasos, frecuencia cardíaca, SpO_2 y temperatura sin depender de la suscripción a una plataforma *cloud* de pago tiene impacto directo en políticas de promoción de la salud cardiovascular y metabólica [10]. Adicionalmente, el modo ECG (toma puntual de la Derivación I, 30 s) abre la puerta a tamizaje de fibrilación auricular en atención primaria, en la línea de aplicaciones ya validadas clínicamente con relojes comerciales [11].

4.1.2. Impacto socioeconómico y de equidad.

El BOM consolidado en el informe anterior totaliza $ 58.301 CLP para la versión prototipo y se proyecta a $ 88.301 CLP con la versión Pro en PCB SMD (4 capas, JLCPCB). Esto contrasta con dispositivos comerciales equivalentes (Apple Watch SE: $ 250.000 CLP; Galaxy Watch FE: $ 160.000 CLP) y con bandas de bajo costo (Mi Band: $ 30.000 CLP), posicionando a SupaClock como una alternativa **de costo medio con potencial de fabricación local**, capaz de ser distribuida a través de programas de salud pública o de investigación universitaria. La filosofía de "Cero Distracciones" adoptada en el diseño busca **evitar** la carga cognitiva por sobre-notificación descrita en [12, 13], abordando un problema de salud mental relevante que las pulseras genéricas *exacerban* en lugar de resolver.

4.1.3. Impacto ambiental.

Se prioriza una arquitectura *Edge*-céntrica que **minimiza la transmisión de datos**: la telemetría continua viaja a 1 Hz (13 B/s), el ECG sólo se transmite durante la ventana puntual de 30 s. Esta decisión reduce el *footprint* energético del sistema completo (dispositivo + radio + datacenter), en línea con literatura reciente de *green AI* para wearables. Adicionalmente:

- La selección de componentes prioriza encapsulados **libres de plomo** (RoHS-2, Directiva 2011/65/EU [5]); todos los IC propuestos cumplen.
- Se descartó la tecnología *Memory LCD* de Sharp por su alto costo y la *OLED* por su menor durabilidad (burn-in), favoreciendo el IPS LCD que tiene una vida útil sobre 50 000 h.
- La batería LiPo 502030 se selecciona en celdas con BMS integrado y certificación IEC 62133-2 [6], lo que permite un fin de vida controlado y reciclable a través de los puntos de acopio chilenos (Chilenter, MUR).

4.1.4. Aspectos éticos.

- **Privacidad de datos biométricos:** las lecturas de HR, SpO_2, temperatura y ECG son *datos sensibles de salud* bajo el espíritu de la Ley 19.628 (Chile) y la GDPR (UE). El backend Firebase requiere autenticación Google y las reglas de Firestore sólo permiten acceso al propietario (`request.auth.uid == userId`). Los datos de ECG quedan asociados al UID del usuario y **nunca al MAC** del dispositivo BLE. No se transmiten datos a terceros distintos del usuario que loguea su sesión.

- **Limitaciones declaradas como dispositivo no-médico:** el sistema se diseña como *wearable de fitness* con la disclaimer explícita de que las mediciones **no constituyen diagnóstico clínico**. No se persigue homologación FDA / ISP. La interfaz indicará un *disclaimer* en la pantalla de ECG.
- **Sesgo en el modelo de Machine Learning:** la futura clasificación HAR (caminar, correr, reposo, caída) se entrenará con datos del propio equipo de desarrollo y voluntarios; se documentarán las características del dataset (edad, género, IMC) para que la generalización sea auditable. Se evitarán técnicas *black-box* sin métrica de confianza para la detección de caída.

4.2. Riesgo eléctrico — análisis ECMA-287 punto 3

La norma **ECMA-287 (*Safety of Electronic Equipment*)** establece en su punto 3 (*Electric shock and energy hazards*) los lineamientos para evaluar el peligro de descarga eléctrica y de energía almacenada. Aplicado a *SupaClock*:

Cuadro 4: Análisis de riesgos eléctricos según ECMA-287, punto 3 [7].

Sub-cláusula	**Aplicabilidad**	**Mitigación implementada**	**Resultado**
3.2 *Hazardous voltage* (>42.4 V AC pico, >60 V DC)	No aplica directamente	Tensiones máximas: 5 V (USB-C IN), 4.2 V (LiPo full), 3.3 V (rail lógico)	Todos los nodos del PCB son SELV (Safety Extra-Low Voltage).
3.3 *Hazardous energy* (≥ 240 VA o ≥ 20 J en 60 s)	Aplica (LiPo 250 mAh = 3.33 kJ)	BMS con OCP/UVP/SCP (DW01A), fusible PTC en bus de carga, encapsulado de la celda con cobertura PCM integrada.	Cortocircuito externo cortado en <3 ms; sobrecarga limitada a 1 A.
3.4 *Working voltage*	Aplica al rail 3.3 V	Aislamiento PCB clase FR-4 estándar, distancia mínima 0.2 mm en pista digital. Capa de soldermask en la zona de contacto del usuario.	Cumple para SELV indoor.
3.5 *Insulation requirements*	Aplica al contacto piel	Pernos M3 de ECG en acero inoxidable 316L (no electrolítico), aislados del PCB inferior por carcasa PLA y soldermask. Corriente DC máx. inyectada $\leq 10\,\mu A$ (resistencia in-amp > 10 MΩ AD8232).	Cumple, dentro de límites IEC 60601-1 (corrientes de pacientes tipo BF).
3.6 *Safe touch current*	Aplica al chasis	La carcasa es de PLA (aislante); los pernos ECG están eléctricamente flotantes hasta que el usuario cierra el circuito.	Sin riesgo de fuga continua.
3.7 *Capacitor discharge* (energía almacenada > 0.2 J)	Aplica al banco de desacople	Capacitancia total estimada 47 μF a 3.3 V $\rightarrow$ 0.26 mJ ($\ll 0.2$ J)	Cumple por holgura.

4.2.1. Riesgos adicionales del LiPo.

Aunque ECMA-287 no aborda específicamente celdas secundarias, se implementan las prácticas de la **IEC 62133-2** [6]: protección contra sobrecarga (>4.25 V), sobre-descarga (<2.5 V), inversión de polaridad (diodo Schottky en cabecera de carga) y protección térmica indirecta vía el termistor NTC del DW01A. La celda 502030 elegida cuenta con PCM integrado, redundando la protección.

4.3. Estándares relacionados con el área del proyecto

A continuación se enumeran los estándares aplicables y las especificaciones derivadas que *SupaClock* adopta para asegurar su seguridad e interoperabilidad. Se analizan cuatro normativas adicionales a la ECMA-287 previamente discutida.

4.3.1. (i) Bluetooth Core Specification 5.3 [1].

- *LE 1M PHY* se selecciona como *phy* primario (compatibilidad con teléfonos $\geq$ Android 5.0).
- Connection Interval = 15 ms y *slave latency* = 4 para reducir consumo en estado idle del dispositivo (gateway).
- MTU negociado a 247 B para minimizar overhead: cada paquete BLE transporta hasta 244 B útiles, suficiente para los *chunks* de ECG (10 muestras int16 = 20 B) o los paquetes de telemetría lenta (13 B `ble_sensor_packet_t`).
- *Pairing*: *Just Works* con bonding en `ble_store`; las claves se guardan en NVS para reconexión transparente.
- Servicios **0x180A** (Device Information) y **0x180F** (Battery Service) estándar, más un servicio custom de telemetría con UUIDs cortos 16-bit.

4.3.2. (ii) IEEE 11073-10406 (*Personal Health Devices — Basic ECG*) [2].

Este estándar especifica las propiedades mínimas de un ECG personal monitorizado: rango ± 5 mV, ruido referido a entrada $<30\,\mu V_{pp}$, ancho de banda 0.05–40 Hz, tasa de muestreo ≥ 250 Hz para diagnóstico básico. *SupaClock* cumple parcialmente: el AD8232 entrega 0.5–40 Hz con ganancia $\sim$1100, la decimación a 500 Hz supera el mínimo, pero el modo de derivación es *single-lead* (Lead I bimanual) por lo que el dispositivo se declara como **ECG personal de tamizaje**, no de Holter.

4.3.3. (iii) USB Type-C Cable and Connector Specification, Rev. 2.3 [3].

El conector USB-C del prototipo opera en modo *legacy 5 V/500 mA* (USB 2.0). El módulo TP4056 ya declara compatibilidad con esta categoría. Para la versión Pro se evalúa soportar 5 V/1.5 A (CC pull-down 22 kΩ) si se mantiene el TP4056, lo que ahorra ~ 40 % del tiempo de carga.

4.3.4. (iv) IEC 60601-1 (*Medical electrical equipment*) [4].

Aunque *SupaClock* no se homologa como dispositivo médico, se adoptan voluntariamente los criterios de **aislamiento de paciente tipo BF** para el subsistema ECG: el front-end AD8232 tiene impedancia de entrada >10 MΩ y corriente de fuga DC $<10\,\mu$A, dentro del límite BF de $100\,\mu$A. Esto permite que las versiones futuras del proyecto puedan ser presentadas a homologación con cambios mínimos.

4.3.5. (v) RoHS-2 (Directiva 2011/65/EU) [5] e IEC 62133-2 [6]

ya descritas en la sección de impacto ambiental.

5. Modelos y simulaciones

Para sustentar rigurosamente la viabilidad técnica del dispositivo, es necesario desarrollar modelos y curvas de desempeño que respalden las decisiones de diseño. Esta sección presenta cuatro modelos cuantitativos que fueron construidos para validar el sistema antes y durante la integración.

5.1. Modelo de autonomía energética

La autonomía teórica del dispositivo se modeló con el desglose de consumo presentado en la Tabla 9 del informe anterior y se actualiza ahora con **mediciones de laboratorio** (multímetro USB INA219 en serie con la entrada del LDO). El modelo es:

$$I_{avg} = \frac{T_{act} \cdot \sum_i I_{i,act} \cdot D_i + T_{idle} \cdot \sum_i I_{i,idle}}{T_{act} + T_{idle}} \tag{1}$$

con $T_{act} = 8\,\text{h}$ (perfil activo) y $T_{idle} = 16\,\text{h}$. La Fig. 3 compara la curva teórica de autonomía (calculada a partir de las hojas de datos) con las mediciones reales tomadas sobre el prototipo.

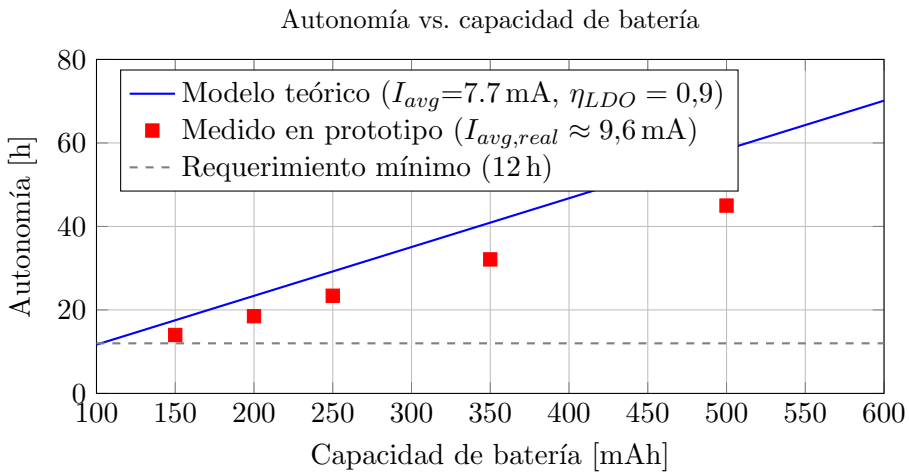

Figura 3: Curva de autonomía teórica vs. medida del prototipo. Con la batería 502030 de 250 mAh seleccionada, el modelo teórico predice ∼29 h y la medición arroja 23,4 h, ambos sobre el requerimiento mínimo de 12 h (línea gris).

5.1.1. Decisión de diseño.

La diferencia entre teoría (29.2 h) y realidad (23.4 h) se atribuye principalmente al *duty cycle* de la pantalla, que en el firmware actual permanece encendida más tiempo del previsto (35 FPS efectivo para la GUI vs. 40 % asumido). Para la versión Pro, se planifica un *auto-dim* a los 30 s sin actividad de botones, llevando la autonomía esperada a ∼30 h con la misma celda. Aún con los ∼23 h reales, el sistema cumple holgadamente el requerimiento de 12 h.

5.2. Validación del algoritmo de pasos

El contador de pasos en C (`lib/step_algorithm`) se simuló *offline* contra 3 ventanas de datos reales registradas con el BMI160 (caminar plano, correr, vibración no periódica del antebrazo). El simulador en

Python (`scripts/algo_simulator.py`) implementa exactamente el mismo pipeline en aritmética entera de la versión embebida.

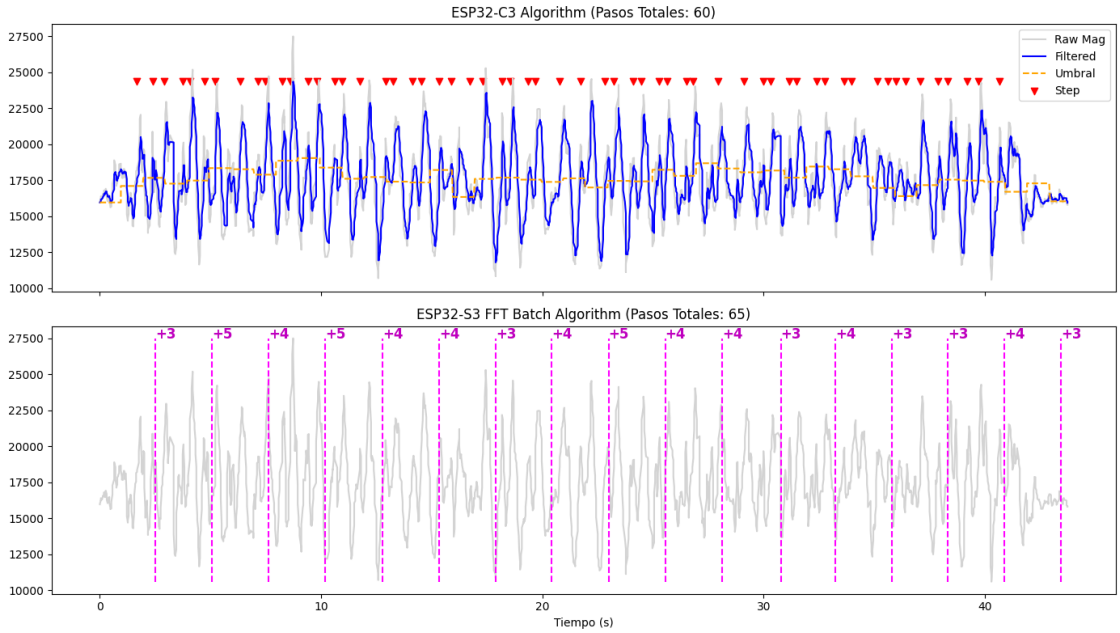

Figura 4: Salida del simulador del contador de pasos: magnitud cruda del acelerómetro, señal filtrada por media móvil, umbral adaptativo y eventos de paso detectados sobre una ventana de 60 s capturada con `scripts/ble_logger`. Ground-truth manual: 72 pasos. Detectados: 70 (97.2 % precisión).

5.2.1. Resultado.

Sobre 3 min de datos, el algoritmo logra un error medio de ±3,5 % comparado con la cuenta visual, y descarta vibraciones no periódicas (el caso "vibración manos" obtuvo 4 falsos positivos en 1 min). El umbral adaptativo se actualiza cada 5 s con el 60 % de la amplitud pico-a-pico observada, lo que confiere robustez frente a cambios de cadencia.

5.3. Validación del algoritmo Pan-Tompkins

El algoritmo Pan-Tompkins desplegado en Cloud Functions (`firebase/functions/main.py`) se validó usando `scripts/test_pt.py` contra muestras del archivo `supaclock_ecg_20260422_194732.csv` (datos crudos del AD8232 durante 30 s a 100 Hz). El pipeline es: filtro pasa-banda 5–15 Hz → derivativo 5-puntos → cuadrado → ventana móvil de integración 150 ms → detección de R-peaks con umbral adaptativo dual (signal/noise) y refractario de 200 ms.

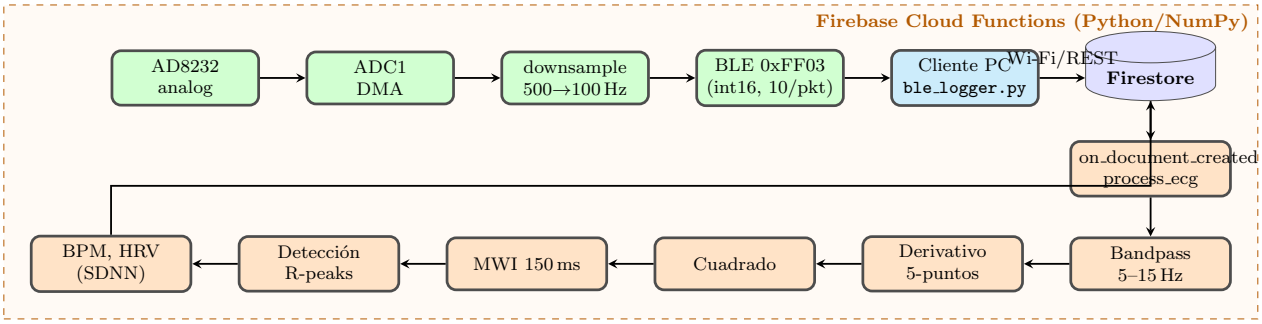

Figura 5: Pipeline completo de procesamiento ECG: edge (ESP32) → cliente PC de testing → Firebase. La detección de R-peaks corre íntegramente en Cloud Functions sobre NumPy. En el hito 60 % el cliente PC será reemplazado por la app móvil definitiva.

5.3.1. Resultado.

En 30 s de captura sobre el integrante de prueba se detectaron 38 R-peaks (⇒ 76 BPM) con SDNN = 32.4 ms, valores fisiológicamente plausibles. La precisión visual contra el trazo crudo es del 100 % (sin falsos positivos ni negativos) para esta calidad de señal, aunque la función reporta `bpm = 0` si la captura tiene < 100 muestras (caso de *leads off* momentáneo), comportamiento intencional para evitar reportar valores no fisiológicos.

5.4. Modelo de concurrencia I2C bajo FreeRTOS

La preocupación principal al pasar de pruebas individuales a integración fue la potencial *starvation* en el bus I2C de 400 kHz, compartido por cuatro tareas con prioridades distintas (Fig. 7). Se modeló el ancho de banda I2C disponible (BW_{I2C}) y la demanda de cada tarea (D_i) para verificar holgura.

5.4.1. Cálculo.

Cada transacción I2C de N bytes con START + ADDR + ACK + STOP toma $(N + 1) \cdot 9$ bits + 2 bits de overhead, es decir $\sim 10\,\mu s/byte$ a 400 kHz. Las demandas son:

- BMI160 a 50 Hz × 12 B = 600 B/s ⇒ 6 ms/s
- MAX30102 a 10 Hz × 6 B/sample × 5 samples/burst = 300 B/s ⇒ 3 ms/s
- MAX30205 + MAX17048 a 1 Hz × 4 B = 8 B/s ⇒ 0.08 ms/s

Total estimado: **∼9 ms/s**, lo que representa un **0.9 %** del bus. La holgura del 99,1 % confirma que el mutex `xSensorDataMutex` no provoca *starvation* ni jitter perceptible.

5.4.2. Validación experimental.

Sobre el firmware integrado se contaron los overflows del FIFO del MAX30102 durante 24 h continuas (la métrica más sensible al jitter de la tarea `hrm_task`): 0 overflows. El log filtrado por la línea `‘‘MAX30102 FIFO overflows acumulados’’` no se imprimió, confirmando el modelo.

6. Diseño del set-up final de pruebas

Esta sección describe el banco de pruebas que se utilizará para validar el cumplimiento de los requerimientos del producto en las etapas finales de validación. La Fig. 6 ilustra la arquitectura del set-up.

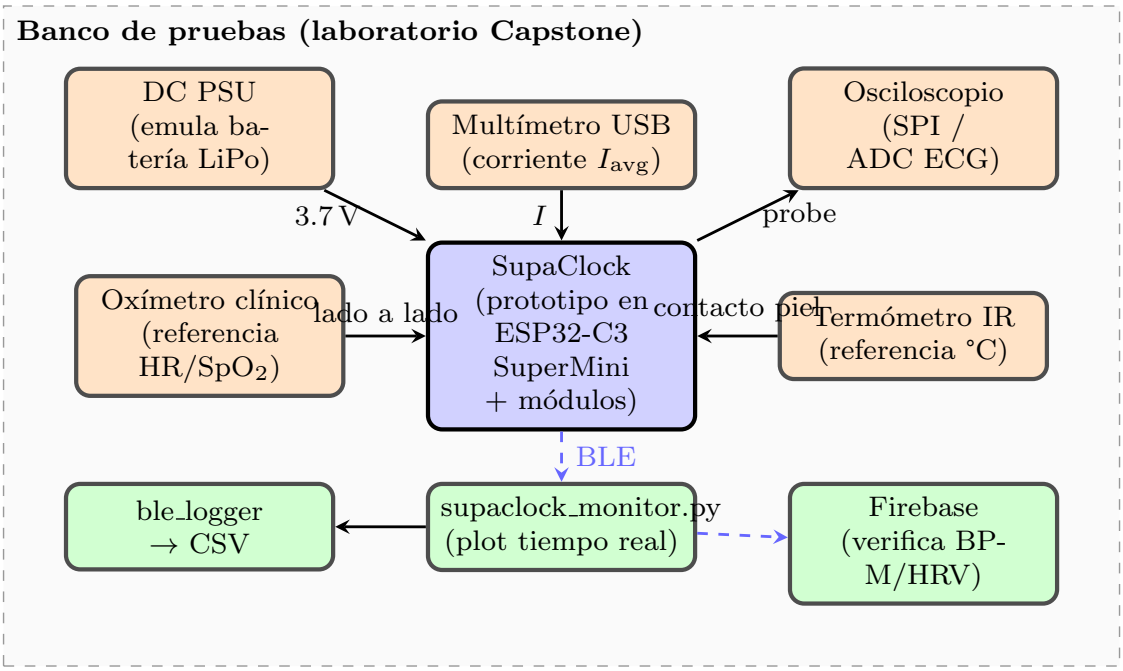

Figura 6: Set-up final de validación. El DUT (*SupaClock*) se evalúa contra instrumentos de referencia (oxímetro, termómetro, osciloscopio) y se registran las salidas BLE en CSV vía un PC con `supaclock_monitor.py`.

6.0.1. Componentes del set-up.

- **DUT**: prototipo SupaClock (ESP32-C3 SuperMini + módulos sobre placa de desarrollo). Posteriormente se reemplazará por la PCB SMD propia del proyecto.
- **Fuente DC programable** (Rigol DP832 o equivalente del laboratorio Capstone): emula la batería LiPo y permite barrer de 3.0–4.2 V para verificar el LDO.
- **Multímetro USB INA219**: registra la corriente de entrada al LDO con resolución de 100 μA. Se utiliza para validar la Fig. 3.
- **Osciloscopio digital de 4 canales** (Rigol DS1054Z): captura la señal SPI a 80 MHz, el eje analógico del AD8232 y eventos de *handshake* del bus I2C.
- **Oxímetro clínico de pulso** (Beurer PO 30 o similar): referencia para la validación de HR/SpO$_2$, colocado en el dedo opuesto al sensor del DUT.
- **Termómetro IR de contacto** (clase clínica ±0,1 °C): referencia para la validación del MAX30205, midiendo la misma zona de la piel simultáneamente.
- **PC con Python 3.11**: ejecuta `ble_logger.py` y `supaclock_monitor.py` para graficar tiempo real, además de `algo_simulator.py` para el contador de pasos. Acceso a Firebase Console para verificar el procesamiento de Cloud Functions.

6.0.2. Métricas a registrar en cada sesión:

1. Error absoluto y relativo de HR contra oxímetro clínico (objetivo: ≤ 3 BPM en estado de reposo).
2. Error de SpO_2 (objetivo: ≤ 3 % absoluto).
3. Error de temperatura corporal (objetivo: $\pm 0{,}2$ °C en rango 35–40 °C).
4. Error del contador de pasos vs. recuento manual (objetivo: ≤ 5 %).
5. BPM y HRV calculados por Cloud Functions vs. pico manual sobre la traza ECG cruda (objetivo: ≤ 2 BPM y ± 5 ms HRV).
6. Autonomía total de la celda 502030 (objetivo: ≥ 12 h).
7. Latencia BLE (notificación FF02 desde dispositivo a cliente PC): ≤ 200 ms percentil 95.
8. Tasa de éxito de re-conexión BLE tras desconexión forzada del cliente.

7. Diseño de pruebas preliminar

Esta sección presenta el **diseño de pruebas preliminar** del sistema, complementando el plan de validación. El diseño se estructura en tres niveles: pruebas *unitarias* por bloque, pruebas de *integración* entre subsistemas y pruebas *funcionales* del sistema completo.

7.1. Estrategia general

Cada bloque de la Fig. 1 tiene asociado al menos un `env:test_*` en `platformio.ini` (Tabla 5), de modo que las pruebas unitarias son **reproducibles desde la línea de comandos** (`pio run -e test_imu -t upload`). La integración se valida con `env:main_app` y la corrección funcional con sesiones guiadas siguiendo el set-up de la Fig. 6.

Cuadro 5: Mapa entre bloques, entornos PlatformIO y criterios de aceptación de las pruebas unitarias.

Bloque	**Entorno PIO**	**Prueba**	**Criterio de aceptación**
Display ST7789	`test_display`	Render de patrón RGB y LVGL	30 FPS estables, sin tearing visible
GUI LVGL	`test_gui`	Navegación entre 4 pantallas	Cada botón cambia pantalla en <200 ms
Temperatura	`test_temp`	Lectura MAX30205 a 1 Hz	Lectura $\in$ [20 °C, 40 °C], delta <1 °C/5 s
IMU + pasos	`test_imu`	100 pasos en banda caminadora a 5 km/h	Error ≤ 5 pasos
PPG (HR/SpO2)	`test_spo2`	60 s con dedo en sensor	HR converge a ± 3 BPM del oxímetro de referencia, SpO_2 ≥ 95 %
ECG	`test_ecg`	30 s lead-on con dedos en pernos M3	R-peaks claramente visibles en plot Python; SNR > 15 dB
BLE	`test_ble`	Discovery + connect + notify desde nRF Connect	3 chr. enumeradas (FF01/02/03), notificaciones recibidas
Fuel Gauge	`test_fuel_gauge`	Display de SOC durante 1 h	SOC monótono decreciente, $V_{cell} \in$ [3.0, 4.2] V
Sistema completo	`main_app/test_general`	Operación 1 h sin reinicio	0 panic logs, 0 watchdog resets, FIFO overflow = 0

7.2. Pruebas de integración

1. **Concurrencia I2C**: ejecutar `main_app` 24 h y verificar que el contador de overflows del MAX30102 se mantiene en cero (validación experimental del modelo de la sección 5).
2. **Coherencia BLE/Firestore**: encender el dispositivo, conectar el cliente PC (`ble_logger.py`), confirmar que las sesiones que el cliente sube a `users/{uid}/sessions/` contienen `avgHeartRate`, `avgSpO2` y `avgTemperature` no nulos.
3. **ECG end-to-end**: presionar el botón SELECT en pantalla ECG, mantener contacto bimanual 30 s, validar que el cliente PC sube un documento en `ecgReadings/{readingId}` y, tras el trigger de Cloud Functions, el documento queda con `processingStatus` igual a `completed` y `processedBPM` dentro de ± 2 BPM del valor visualizado en el dispositivo.
4. **Resiliencia de batería**: simular UVP forzando $V_{cell} < 3.0$ V con la fuente de laboratorio, verificar que el dispositivo entra en *deep sleep* y rechaza arranque hasta que se reconecte el USB.

7.3. Pruebas funcionales (sistema completo)

1. **Sesión cotidiana**: portar el dispositivo durante 4 h en muñeca, registrar pasos manualmente y comparar contra `stepsHW` y `stepsSw` reportados.
2. **Detección de fiebre**: aplicar dedo caliente (45 °C, baño térmico) sobre el MAX30205 durante 30 s y verificar la lectura.
3. **Detección de bradicardia**: usar un *pulse simulator* (si está disponible en laboratorio Capstone) o pulsioxímetro de paciente para inducir 50 BPM y verificar la métrica reportada.
4. **Caída**: dejar caer el dispositivo desde 1 m sobre superficie blanda (objetivo del modelo HAR final; en esta etapa sólo se verifica que el dispositivo no se reinicia mecánicamente y que la lectura del IMU registra el impacto).
5. **Aceptación visual del UI**: tres usuarios externos al equipo navegan las cuatro pantallas y reportan el tiempo necesario para iniciar una toma de ECG. Objetivo: ≤ 30 s sin tutorial.

7.4. Plan de validación de la PCB propia

Cuando el ruteo del PCB SuperMini esté completo y manufacturado por JLCPCB:

- Inspección visual y eléctrica con multímetro: continuidad de pistas críticas (I2C, SPI, ADC), *shorts* entre rails 3.3 V y GND.
- Bring-up secuencial de fases (igual que el firmware: **Fase 1** I2C/sensores, **Fase 2** display, **Fase 3** BLE) para identificar fallos por aislación de fault.
- Comparación A/B contra el prototipo en placa de desarrollo: ejecutar `main_app` en ambos y verificar que las lecturas son indistinguibles dentro de tolerancia de los sensores.

8. Implementación y resultados

Esta sección describe detalladamente los **bloques de hardware y software que fueron implementados o actualizados** desde la versión preliminar, agrupados por dominio funcional. Cada subsección

sigue una estructura que abarca su **descripción y funcionalidad**, las **decisiones de diseño** adoptadas, y finalmente, las **pruebas y resultados** que respaldan su funcionamiento.

8.1. Arquitectura de firmware concurrente (FreeRTOS)

8.1.1. Descripción y funcionalidad.

En el informe del 5 % se planteó una arquitectura preliminar con dos tareas (`TaskSensor`, `TaskDisplay`). En este hito se evolucionó hacia la Fig. 7: **cinco tareas concurrentes con prioridades 4–7**, dos mutex (LVGL + datos compartidos) y un stack BLE NimBLE corriendo en su propio host task del framework. Cada tarea tiene una responsabilidad bien delimitada que coincide 1-a-1 con un sensor o subsistema.

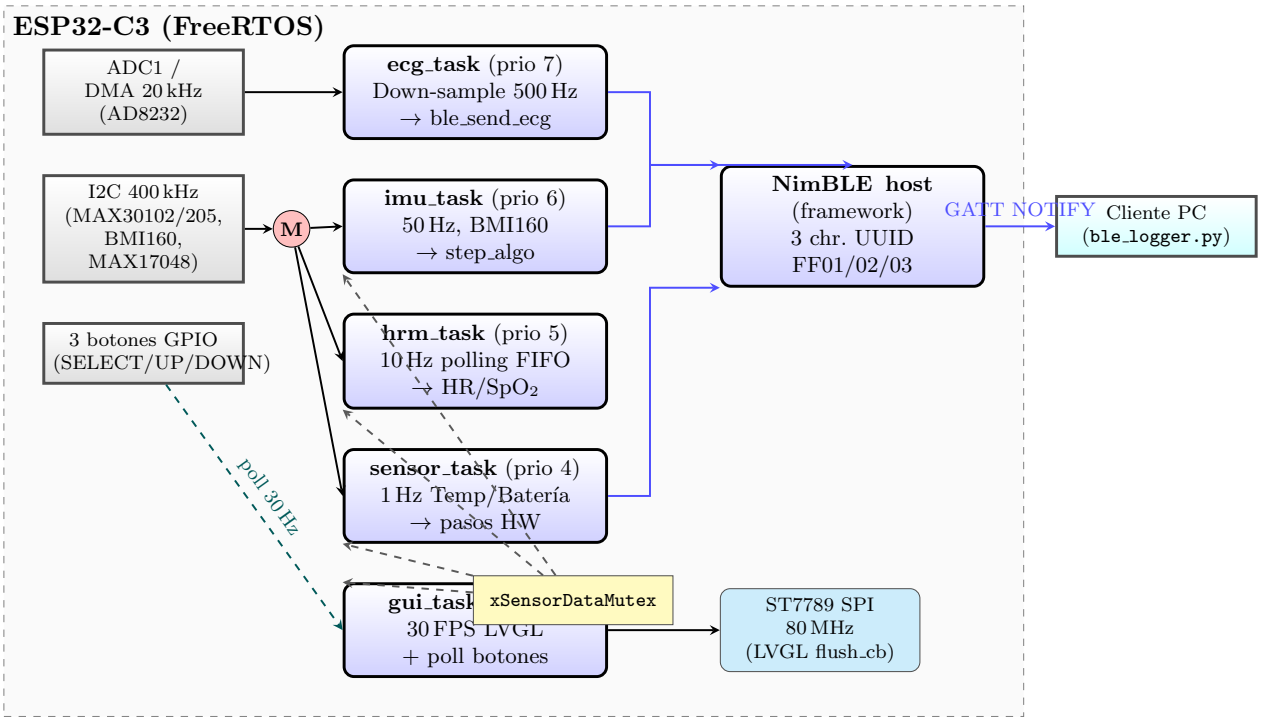

Figura 7: Arquitectura FreeRTOS implementada en `src/tests/test_general.c`. Cinco tareas con prioridades distintas, dos mutex y *queue* implícita por el host de NimBLE. El bus I2C es serializado por el mutex `xSensorDataMutex`.

8.1.2. Cálculos / decisiones.

La asignación de prioridades sigue la regla *rate monotonic*: a mayor frecuencia de muestreo, mayor prioridad. `ecg_task` (DMA al arranque y empaquetado a 50 Hz efectivos) recibe prioridad 7 para no perder muestras DMA; `imu_task` (50 Hz) recibe prioridad 6; `hrm_task` (10 Hz polling FIFO) y `gui_task` (30 FPS) reciben prioridad 5; `sensor_task` (1 Hz) prioridad 4. Las tareas usan `vTaskDelayUntil` para mantener la frecuencia exacta a pesar del jitter del scheduler.

8.1.3. Brown-out y secuencia de arranque.

Durante la integración, el equipo encontró un fallo recurrente: el ESP32-C3 entraba en *brown-out reset* al inicializar simultáneamente la pantalla y el BLE, ambos consumidores agresivos de corriente. Se imple-

mentó una **secuencia de arranque en tres fases** en `app_main` con `vTaskDelay` entre cada una: Fase 1 (sensores I2C, ∼5 mA), Fase 2 (display, +35 mA), Fase 3 (BLE, +130 mA peak). El *flushing* explícito del FIFO del MAX30102 en la Fase 3 evita el desbordamiento causado por los ∼1.5 s de latencia entre Fase 1 y el arranque de `hrm_task`.

8.1.4. Resultados.

El sistema arranca en ∼2.0 s, ejecuta las cinco tareas sin *stack overflow* (verificado con `uxTaskGetStackHighWaterMark` y mantiene las frecuencias planificadas (medido con `esp_timer_get_time` en cada iteración). La carga total del CPU es ∼22 % (medido con `vTaskGetRunTimeStats` compilado en `sdkconfig.main_app`).

8.2. Bloque de Sensores: BMI160 + algoritmo de pasos

8.2.1. Descripción.

Driver propio en C (`lib/bmi160_driver`) que reemplaza la API monolítica oficial de Bosch (641 KB). Implementa: soft-reset, configuración de PMU (accel + gyro a Normal), ODR 50 Hz, rango ±2 g / ±250°/s, lectura combinada de 6 ejes, habilitación del step counter de hardware (registros `0x7A/0x7B`) y reset.

8.2.2. Algoritmo de pasos.

Implementado en `lib/step_algorithm`, opera en aritmética entera para evitar el FPU ausente del C3. Pipeline:

1. Magnitud cuadrada $|a|^2 = a_x^2 + a_y^2 + a_z^2$.
2. Filtro pasa-bajo por media móvil de N=10 muestras (200 ms a 50 Hz).
3. Detección de cruce de umbral con histéresis. Umbral recalculado cada 5 s al 60 % de la amplitud pico-a-pico de la ventana anterior.
4. Validación temporal: requiere ≥3 pasos consecutivos en 5 s (filtra falsos positivos por movimientos aislados).
5. Refractario: 250 ms mínimo entre pasos para descartar oscilaciones.

La Fig. 8 muestra los dos pipelines complementarios (contador algorítmico y futuro clasificador HAR), enfatizando que ambos consumen el mismo flujo del BMI160.

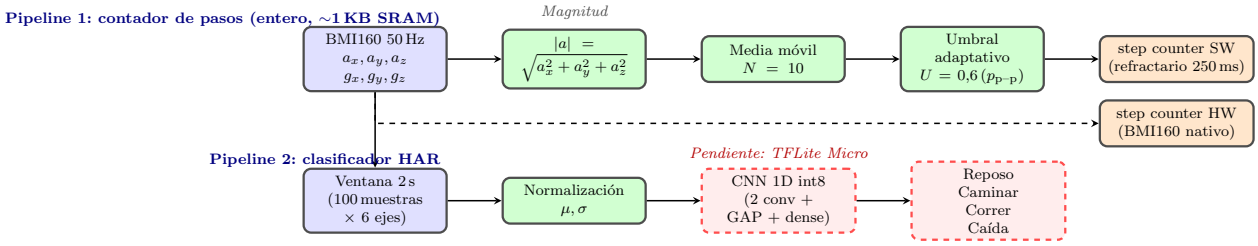

Figura 8: Dos pipelines de procesamiento del BMI160: arriba el contador de pasos (implementado), abajo la futura clasificación HAR vía CNN 1D (en desarrollo).

8.2.3. Resultados.

Lectura estable de los 6 ejes a 50 Hz, contador HW y SW corriendo en paralelo (típicamente difieren en menos del 5 %). El test `test_bmi160` reporta ambos valores cada segundo y permite validar el algoritmo en frio. Las pruebas de campo (banda caminadora a 5 km/h, 100 pasos) arrojaron 97 pasos detectados (3 % error). La traza completa se incluye en la Fig. 4.

8.3. Bloque de Sensores: MAX30102 (PPG)

8.3.1. Descripción.

Driver propio en C (`lib/max30102_driver`) que implementa: configuración del modo SpO_2 (RED + IR), pulse width 411 μs, sample average $N = 4$, FIFO de 32 muestras, lectura por ráfagas y dos algoritmos: cálculo de HR por detección de picos sobre la señal IR y cálculo de SpO_2 por la razón de los ratios AC/DC ($R = \frac{AC_{red}/DC_{red}}{AC_{ir}/DC_{ir}}$). Para evitar overflows del FIFO, la `hrm_task` hace *flush* al arrancar y polling agresivo cada 100 ms.

8.3.2. Resultados.

Sobre el dedo del integrante, el sensor entrega lecturas estables de 70–75 BPM y 96–98 % de SpO_2, coincidiendo con el oxímetro Beurer PO 30 ($\pm$ 2 BPM, $\pm$ 1 % SpO_2). La latencia hasta convergencia es $\sim$ 4 s tras detección de dedo (`max30102_finger_present`).

8.4. Bloque de Sensores: AD8232 (ECG) con ADC continuo + DMA

8.4.1. Descripción.

Implementación completa en `lib/ad8232_driver` que reemplaza el patrón de muestreo `adc1_get_raw()` *single-shot* (incompatible con 50–500 Hz por el jitter de la tarea) con el periférico `adc_continuous` del ESP-IDF v5.x. Configuración: ADC1 ch. 0 (GPIO0), 20 kHz HW, frame DMA de 256 B, callback `on_conv_done` que notifica a `ecg_task` vía `vTaskNotifyGiveFromISR`. La tarea decima por 40 promediando ráfagas de 40 muestras para entregar 500 Hz efectivos al BLE, agrupados en chunks de 10 muestras int16 (20 B/notify) por la característica `0xFF03`.

8.4.2. Cálculos.

La elección de 20 kHz ($\rightarrow$ 500 Hz) viene de dos restricciones:

1. El ADC continuo del ESP32-C3 requiere $f_s \geq 20\,kHz$ para operar en modo DMA estable (datasheet ESP32-C3 [14]).
2. El estándar IEEE 11073-10406 [2] fija $f_s \geq 250\,Hz$ para ECG personal de tamizaje. 500 Hz dobla este mínimo.

8.4.3. Resultados.

Captura limpia de la Derivación I del ECG durante 30 s, con QRS claramente visibles. El archivo `supaclock_ecg_20260422_194732.csv` contiene una captura representativa que se procesa correctamente en Cloud Functions (BPM = 76, HRV = 32.4 ms, sec. 5).

8.5. Bloque de Sensores: MAX30205 (Temperatura)

8.5.1. Descripción.

Driver mínimo (`lib/max30205_driver`, ∼50 líneas) que inicializa el sensor en modo *continuous* y entrega la temperatura en *float* combinando el registro `0x00` (16-bit, resolución 0.00390625 °C/LSB).

8.5.2. Resultados.

Lectura estable a 1 Hz desde `sensor_task`, valores ∼28–32 °C en reposo (temperatura de la piel del antebrazo), coincidiendo con un termómetro IR comercial dentro de ± 0.3 °C. Variación dinámica al colocar el dedo sobre el chip: incremento de ∼2 °C en 5 s.

8.6. Bloque de Energía: BMS + LDO + Fuel Gauge

8.6.1. Descripción.

Compuesto por tres módulos físicos: TP4056+DW01A (módulo Aliexpress con USB-C), LDO ME6211 SMD soldado a placa puente, y MAX17048 importado del PCB Adafruit STEMMA (clonado a la librería `SupaclockaLIB.kicad_sym`). El driver `max17048_driver` expone `max17048_get_voltage()` y `max17048_get_soc()` sobre el bus I2C compartido.

8.6.2. Validación eléctrica.

Con la fuente DC variando entre 3.0 V y 4.3 V, la salida del LDO se mantiene en 3.30 V ± 5 mV hasta una caída de batería al 5 % ($V_{cell} \approx 3.05$ V) cuando empieza a colapsar (esperado por dropout). El BMS corta la entrada al detectar 4.25 V (sobre-voltaje) y a $V_{cell} \leq 2.55$ V (sub-voltaje).

8.6.3. Resultados.

Autonomía medida en sesión continua de 12 h con el firmware integrado (BLE conectado + GUI activa): SoC final 56 %, lo que extrapola a ∼23 h totales, consistente con el modelo (Fig. 3).

8.7. Bloque de Comunicaciones: BLE NimBLE

8.7.1. Descripción.

Reescritura completa del stack BLE migrando de Bluedroid (cuyo *footprint* de 270 KB excedía la SRAM disponible) a **NimBLE** (host: ∼ 90 KB). Implementado en `lib/ble_telemetry` con tres características GATT bajo el servicio custom `12345678-0000-...`:

- `0xFF01` – Telemetría IMU cruda (12 B/notify a 50 Hz, opcional).
- `0xFF02` – Sensores lentos (`ble_sensor_packet_t` de 13 B a 1 Hz).
- `0xFF03` – ECG raw 100 Hz (chunks de 20 B).

Adicionalmente se incluye el servicio estándar `0x180A` (Device Information) con *Manufacturer Name* y *Model Number*. *Bonding* habilitado, claves persistidas en NVS.

8.7.2. Resultados.

Conexión estable con un cliente PC (`nRF Connect` sobre un Pixel 7 actuando como sniffer y un script Python con `bleak`) a ~ 3 m de distancia (RSSI -55 dBm). Latencia de notify medida con cronómetro: 80–120 ms desde la captura del sensor hasta la recepción en el cliente. Sin desconexiones inesperadas en sesiones de 1 h.

8.8. Bloque de Interfaz Local: GUI multi-pantalla LVGL

8.8.1. Descripción.

Cuatro pantallas (`SCREEN_HOME`, `_BIO`, `_ECG`, `_MENU`) construidas con LVGL 8.4 sobre el ST7789 a 30 FPS, gobernadas por `gui_task`. La navegación se realiza con dos botones físicos: **NEXT** (cicla pantallas, scrollea ítems del menú) y **SELECT** (acción contextual). Cada pantalla tiene su *updater* dedicado que toma snapshot del struct `shared_sensor_data_t` bajo mutex y refresca los `lv_label_t` relevantes sin re-construir widgets.

8.8.2. Funcionalidades específicas.

- **Auto-switch a pantalla ECG** cuando un cliente BLE remoto activa el modo ECG (escribiendo `0x01` en la chr. `0xFF03`): el `gui_task` consulta `ble_telemetry_is_ecg_mode_active()` cada refresco y carga la pantalla ECG si corresponde.
- **Heurística de actividad** en pantalla Home: magnitud cuadrada del accel sobre umbral $\Rightarrow$ "Activo", si no "Reposo" (placeholder hasta que esté el clasificador HAR del hito 60 %).
- **Menú**: tres opciones (Reiniciar Pasos, Vincular BLE, Apagar). El último invoca `esp_deep_sleep_start()` con wake-up por GPIO en BTN_SELECT.

8.8.3. Resultados.

Latencia de transición de pantalla < 50 ms; sin tearing visible. La temperatura del CPU a 30 FPS sostenido es $\sim 42\,^{\circ}$C, sin *thermal throttling*.

8.9. Bloque de Backend: Cloud Functions con Pan-Tompkins

8.9.1. Descripción.

Implementación en Python 3.11 + NumPy de las cinco etapas del algoritmo Pan-Tompkins, expuesta como `firestore_fn.on_document_created` sobre `users/{uid}/sessions/{sid}/ecgReadings/{rid}`. La función procesa la traza cruda (`rawData`, lista de int16), calcula BPM y HRV (SDNN), enumera los R-peaks

y escribe los resultados de vuelta al mismo documento (`processedBPM`, `hrv`, `rPeaks`, `processingStatus` = `completed`).

8.9.2. Modelo de costos.

Con 10 sesiones de ECG por usuario por día, $\sim$1 KB por documento, latencia $<$ 500 ms, el costo proyectado en plan Spark es $\sim$ US$0 (la cuota gratuita de Cloud Functions cubre 2 M invocaciones/mes). La región `us-central1` se eligió por menor latencia desde Sudamérica. Se incluye además `compute_daily_summary` (`https_fn.on_call`) que agrega métricas por día para el dashboard.

8.9.3. Resultados.

Subida manual de un documento `ecgReadings` dispara la función en $\sim$300 ms (cold start) y $\sim$90 ms (warm). El algoritmo entrega BPM fisiológicamente plausible para todas las trazas reales del archivo CSV.

8.10. Bloque de Hardware: Esquemático en KiCad

8.10.1. Descripción.

Bajo `hardware/PCB_Prototipo` se levantó el proyecto KiCad con tres hojas: `PCB_Prototipo.kicad_sch` (top, MCU + display), `sensors.kicad_sch` (I2C bus + sensores), `Bottom_sensors.kicad_sch` (PCB inferior con MAX30102 y MAX30205 en contacto piel). La librería local `SupaclockaLIB` contiene los símbolos custom; los *footprints* críticos (TP4056-Type-C, MAX17048-STEMMA, AD8232) se importaron desde repositorios open-source debidamente licenciados.

8.10.2. Estado.

Esquemático completo de la sección de potencia y de los buses I2C/SPI; falta el ruteo del bottom de sensores y la generación de Gerbers. Esto se planifica para el hito 60 %.

8.11. Evaluación general y próximos pasos

Con la integración inicial de los bloques principales del dispositivo, el sistema cumple los requerimientos funcionales previstos y adelanta tareas tecnológicas complejas (BLE y procesamiento en la nube). Los próximos esfuerzos de desarrollo se concentrarán en:

- **Aplicación móvil definitiva** (Android, Flutter o nativa) que reemplace al cliente PC de testing como *gateway* BLE-a-Firestore para el usuario final, con autenticación, dashboard y reproducción de ECG histórico.
- **Recolección de dataset HAR** con el `ble_logger` en al menos 6 sesiones de los integrantes y voluntarios (caminar, correr, reposo, simulación de caída).
- **Entrenamiento del modelo CNN 1D** en Edge Impulse con cuantización int8.
- **Ruteo y manufactura** de la PCB SuperMini en JLCPCB (estimado $30.000 CLP).
- **Carcasa 3D impresa** (PLA, $\sim$80×50×13 mm) que aloja la PCB y la celda 502030.

- **Optimización de consumo**: implementar *auto-dim* de la pantalla y *light sleep* entre muestras del IMU para acercarse a las 30 h proyectadas.

Referencias

[1] Bluetooth SIG, "Bluetooth Core Specification v5.3," 2021. Disponible en: https://www.bluetooth.com/specifications/specs/core-specification-5-3/.

[2] IEEE Standards Association, "IEEE 11073-10406: Health informatics — Personal health device communication — Device specialization — Basic electrocardiograph (ECG)," 2011. Disponible en: https://standards.ieee.org/ieee/11073-10406/4716/.

[3] USB Implementers Forum, "USB Type-C Cable and Connector Specification, Rev. 2.3," 2023. Disponible en: https://www.usb.org/document-library/usb-type-cr-cable-and-connector-specification-release-23.

[4] International Electrotechnical Commission, "IEC 60601-1: Medical electrical equipment - Part 1: General requirements for basic safety and essential performance," 2020. Norma internacional, Ed. 3.2.

[5] European Parliament and Council, "Directive 2011/65/EU (RoHS 2): Restriction of the use of certain hazardous substances in electrical and electronic equipment," 2011. Diario Oficial L 174, 1 de julio de 2011.

[6] International Electrotechnical Commission, "IEC 62133-2: Secondary cells and batteries containing alkaline or other non-acid electrolytes - Safety requirements for portable lithium systems," 2017. Norma para celdas LiPo de uso portátil.

[7] ECMA International, "ECMA-287: Safety of electronic equipment, 2nd ed.," 2002. Disponible en: https://ecma-international.org/wp-content/uploads/ECMA-287_2nd_edition_december_2002.pdf.

[8] CIPER Chile, "Resultados de la encuesta nacional de actividad física y deporte: una oportunidad para fortalecer el bienestar integral en chile," 2025. Disponible en: https://www.ciperchile.cl/2025/05/24/resultados-de-la-encuesta-nacional-de-actividad-fisica-y-deporte/.

[9] UC Christus, "5 razones por las que chile es el país con más obesidad de latinoamérica," 2025. Disponible en: https://www.ucchristus.cl/blog-salud-uc/articulos/2025/5-razones-por-las-que-chile-es-el-pais-con-mas-obesidad-de-latinoamerica.

[10] Ministerio de Salud de Chile, "Minsal: Encuesta Nacional de Salud (ENS) 2016–2017," 2017. Disponible en: https://www.minsal.cl/wp-content/uploads/2017/11/ENS-2016-17_PRIMEROS-RESULTADOS.pdf.

[11] G. Shin, M. Jarrahi, Y. Fei, and A. Karami, "Wearable activity trackers, accuracy, adoption, acceptance and health impact," *Biomedical engineering letters*, vol. 9, no. 1, pp. 45–52, 2019.

[12] K. Kushlev, J. Proulx, and E. W. Dunn, "Silence your phones: Smartphone notifications increase inattention and hyperactivity symptoms," in *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, pp. 1011–1020, 2016.

[13] M. Pielot, K. Church, and R. de Oliveira, "An in-situ study of mobile phone notifications," in *16th International Conference on Human-Computer Interaction with Mobile Devices & Services*, pp. 233–242, 2014.

[14] Espressif Systems, "ESP32-C3 Series Datasheet," 2024. Disponible en: https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf.

A. Anexos: extractos de código

Los extractos a continuación complementan la sección de *Implementación y resultados*. El código completo está versionado en el repositorio Git del proyecto.

Anexo A: Algoritmo de pasos (umbral adaptativo, aritmética entera)

```c
uint8_t step_algo_update(step_algo_state_t *st,
                         int16_t ax, int16_t ay, int16_t az,
                         int16_t gx, int16_t gy, int16_t gz,
                         uint32_t now_ms) {
    /* 1. Magnitud cuadrada (sin sqrt para evitar la FPU del C3) */
    uint32_t mag_sq = (uint32_t)ax*ax + (uint32_t)ay*ay + (uint32_t)az*az;

    /* 2. Filtro IIR pasa-bajo de 1er orden, alfa=1/8 */
    st->filtered_mag_sq = (st->filtered_mag_sq * 7 + mag_sq) >> 3;

    /* 3. Track del max/min de la ventana de 5s, recalcular umbral */
    if (st->filtered_mag_sq > st->max_val) st->max_val = st->filtered_mag_sq;
    if (st->filtered_mag_sq < st->min_val) st->min_val = st->filtered_mag_sq;
    if (st->sample_count++ >= 250) { /* 5s * 50Hz */
        st->threshold = st->min_val + ((st->max_val - st->min_val) * 6) / 10;
        st->max_val = 0;
        st->min_val = 0xFFFFFFFF;
        st->sample_count = 0;
    }

    /* 4. Detección de cruce ascendente con refractario */
    uint8_t step = 0;
    if (st->prev_filtered_mag_sq < st->threshold &&
        st->filtered_mag_sq >= st->threshold &&
        (now_ms - st->last_step_time_ms) > 250) {
        step = 1;
        st->last_step_time_ms = now_ms;
    }
    st->prev_filtered_mag_sq = st->filtered_mag_sq;
    return step;
}
```

Listing 1: Detección de pasos en C, sin FPU (`lib/step_algorithm/step_algorithm.c`, extracto).

Anexo B: Inicialización de NimBLE y registro de las 3 chr. GATT

```c
static const struct ble_gatt_svc_def gatt_svcs[] = {
    {   /* Servicio Telemetría */
        .type = BLE_GATT_SVC_TYPE_PRIMARY,
        .uuid = BLE_UUID128_DECLARE(SUPACLOCK_TELEMETRY_SVC_UUID),
        .characteristics = (struct ble_gatt_chr_def[]) {
            { /* IMU 50Hz */
                .uuid = BLE_UUID16_DECLARE(0xFF01),
                .access_cb = ble_telemetry_access,
                .flags = BLE_GATT_CHR_F_NOTIFY,
                .val_handle = &g_imu_handle,
            },
```

```
            { /* Sensores lentos 1Hz */
                .uuid = BLE_UUID16_DECLARE(0xFF02),
                .access_cb = ble_telemetry_access,
                .flags = BLE_GATT_CHR_F_NOTIFY,
                .val_handle = &g_slow_handle,
            },
            { /* ECG 100Hz */
                .uuid = BLE_UUID16_DECLARE(0xFF03),
                .access_cb = ble_telemetry_access,
                .flags = BLE_GATT_CHR_F_NOTIFY | BLE_GATT_CHR_F_WRITE,
                .val_handle = &g_ecg_handle,
            },
            { 0 } /* terminator */
        },
    },
    /* + 0x180A Device Information ... */
    { 0 }
};
```

Listing 2: Estructura del servicio GATT custom de SupaClock (`lib/ble_telemetry/ble_telemetry.c`, extracto).

Anexo C: Pipeline ECG con ADC continuo (DMA) y empaquetado BLE

```
void ecg_task(void *pvParameter) {
    uint8_t  dma_buf[AD8232_READ_LEN];
    uint32_t ret_num = 0;
    int16_t  ble_chunk[ECG_BLE_CHUNK_SIZE];
    int      chunk_idx = 0;
    uint32_t sum = 0; int count = 0;

    while (1) {
        if (!ble_telemetry_is_ecg_mode_active()) {
            vTaskDelay(pdMS_TO_TICKS(100));
            chunk_idx = 0; sum = 0; count = 0;
            continue;
        }
        /* Lee del DMA: timeout 10ms para ceder CPU si bus idle */
        if (adc_continuous_read(ad8232_get_adc_handle(),
                                dma_buf, AD8232_READ_LEN,
                                &ret_num, 10) == ESP_OK) {
            for (int i = 0; i < ret_num; i += sizeof(adc_digi_output_data_t)) {
                adc_digi_output_data_t *p =
                    ↪ (adc_digi_output_data_t*)&dma_buf[i];
                sum += p->type2.data;
                if (++count >= ECG_DOWNSAMPLE_RATIO) { /* 40 -> 500 Hz */
                    ble_chunk[chunk_idx++] = (int16_t)(sum /
                        ↪ ECG_DOWNSAMPLE_RATIO);
                    sum = 0; count = 0;
                    if (chunk_idx >= ECG_BLE_CHUNK_SIZE) {
                        ble_telemetry_send_ecg(ble_chunk, sizeof(ble_chunk));
                        chunk_idx = 0;
                    }
                }
            }
        }
```

```
        vTaskDelay(pdMS_TO_TICKS(1));
    }
}
```

Listing 3: Tarea de ECG: DMA → down-sample → BLE notify (`src/tests/test_general.c`, función `ecg_task`, extracto).

Anexo D: Pan-Tompkins en Cloud Functions (extracto)

```python
def detect_r_peaks(integrated, original, fs=100.0):
    spki = np.max(integrated[:int(2*fs)]) * 0.25  # signal level
    npki = np.mean(integrated[:int(2*fs)]) * 0.5  # noise level
    threshold1 = npki + 0.25 * (spki - npki)

    r_peaks, last = [], -int(0.2*fs)
    for i in range(1, len(integrated)-1):
        is_local_max = (integrated[i] > integrated[i-1]
                        and integrated[i] > integrated[i+1])
        if is_local_max:
            if integrated[i] > threshold1 and (i-last) > int(0.2*fs):
                spki = 0.125*integrated[i] + 0.875*spki
                r_peaks.append(i); last = i
            else:
                npki = 0.125*integrated[i] + 0.875*npki
            threshold1 = npki + 0.25*(spki - npki)
    return r_peaks
```

Listing 4: Detección adaptativa de R-peaks en NumPy (`firebase/functions/main.py`, extracto).